Universidad Nacional Autónoma de México
Facultad de Ciencias



Inteligencia Artificial

Laboratorio 2.13

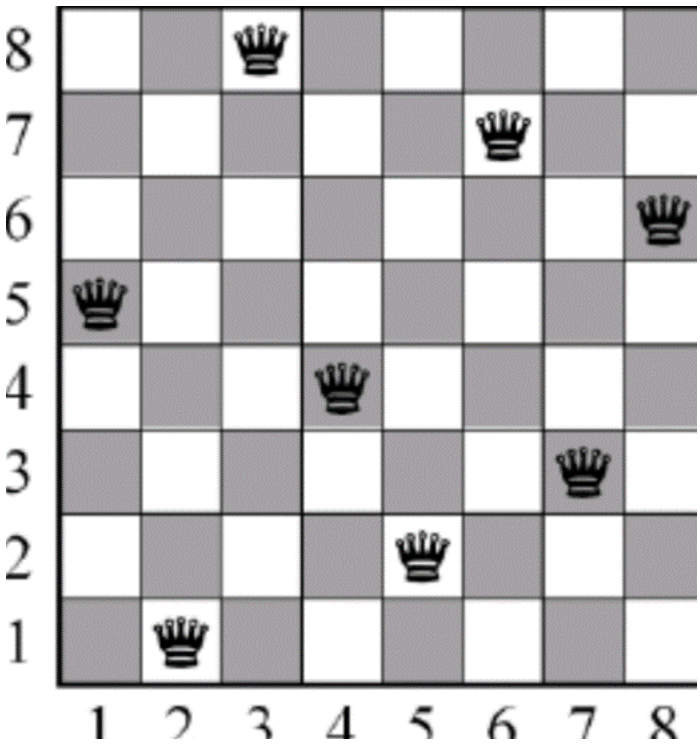
Docente: *Dra. JessicaSarahiMéndezRincón*

Alumna: *OsorioLopezClaudiaYarinet*

Fecha de realización: 21/09/2026

PARTE 4: Problema de las 8 Reinas

Colocar 8 reinas en un tablero de ajedrez sin que se ataquen.



1. Definiciones del Problema

Para explicar esto se puede desglosar así:

- **Estado**: Una lista o vector $L = [c_0, c_1, \ldots, c_7]$ donde cada índice representa una fila y el valor c_i representa la columna donde está la reina. Un estado parcial tiene algunas reinas colocadas; un estado final tiene las 8 sin ataques.

- **Operadores**: Colocar una reina en la siguiente fila vacía ($fila + 1$) en una columna j tal que $0 \leq j < 8$, siempre que no sea atacada por las reinas previas.
- **Función objetivo**: Verificar si hemos llegado a la fila 8 (todas las reinas colocadas) cumpliendo las restricciones de no ataque.Espacio de Búsqueda: Si no usáramos reglas, sería 8^8 (16,777,216 estados). Si restringimos a una reina por columna y fila, baja a $8!$ (40,320 estados).

- **¿Es un problema de satisfacción de restricciones (CSP)?** Sí. Las variables son las posiciones de las reinas, el dominio son las casillas del tablero $\{0...7\}$ y las restricciones son que no pueden compartir fila, columna ni diagonal.
- **¿Cómo cambia si usamos A estrella?** Pasamos de una búsqueda "ciega" (Backtracking/DFS) a una búsqueda informada. A* necesita un costo acumulado $g(n)$ (reinas colocadas) y una heurística $h(n)$. El algoritmo priorizaría expandir ramas que parezcan "más prometedoras".
- **¿Qué heurística podría aplicarse?** Una común es el Número de conflictos potenciales: Contar cuántas casillas quedan "libres" o seguras en las filas restantes. A más casillas libres, mejor es el estado.

Programa

```
import matplotlib.pyplot as plt

def es_seguro(tablero, fila, columna):
    # Verificar si es seguro colocar una reina en la fila, columna
    for i in range(fila):
        if tablero[i] == columna or \
           tablero[i] - i == columna - fila or \
           tablero[i] + i == columna + fila:
            return False
    return True

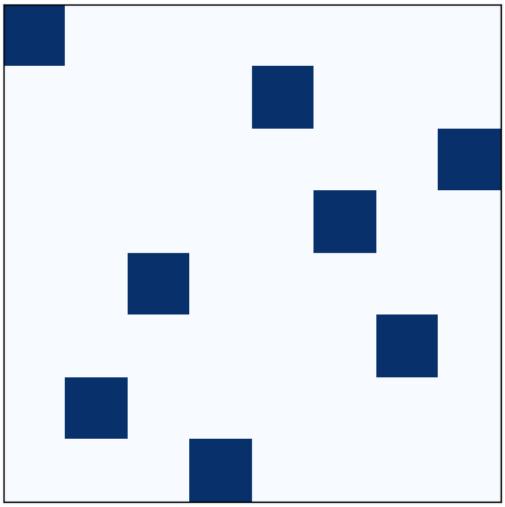
def resolver_8_reinas(tablero, fila):
    if fila == 8:
        return True
    for columna in range(8):
        if es_seguro(tablero, fila, columna):
            tablero[fila] = columna
            if resolver_8_reinas(tablero, fila + 1):
                return True
            tablero[fila] = -1
    return False

def dibujar_tablero(tablero):
    plt.figure(figsize=(8, 8))
    plt.matshow([[tablero[i] == j for j in range(8)] for i in range(8)], cmap="Blues")
    plt.xticks([])
    plt.yticks([])
    plt.show()

# Inicializar el tablero
tablero = [-1] * 8

# Resolver el problema de las 8 reinas
if resolver_8_reinas(tablero, 0):
    print("Solución encontrada:")
    for fila, columna in enumerate(tablero):
        print(f"Reina en fila {fila} y columna {columna}")
    dibujar_tablero(tablero)
else:
    print("No se encontró solución.")
```

```
Solución encontrada:
Reina en fila 0 y columna 0
Reina en fila 1 y columna 4
Reina en fila 2 y columna 7
Reina en fila 3 y columna 5
Reina en fila 4 y columna 2
Reina en fila 5 y columna 6
Reina en fila 6 y columna 1
Reina en fila 7 y columna 3
<Figure size 800x800 with 0 Axes>
```



Preguntas:

1. ¿Es un algoritmo óptimo?
2. Indica la complejidad y espacio de búsqueda con el programa propuesto
3. Indica la limitación del código actual
4. Realiza una mejora sobre el programa y establece el pseudocodigo previo.

Conclusiones finales

Este problema de las 8 reinas ya lo conocia por una tarea de Algebra Superior y me resulto bastante interesante y divertida, aunque estoy aprendiendo a entender y jugar ajedrez

---

© 2026 UNAM Facultado de Ciencias – Todos los derechos reservados